NKT
Tag 3: Interfaces, Such/Sortieralgorithmen, Verkettete Listen

Tobias Wrede, Alexa Katharina-Ronja Brockmöller

5. März 2026

Gliederung

1. Interfaces
2. Listen
3. Binäre Suche
4. Sortieralgorithmen
5. Generics

Gliederung

1. Interfaces
2. Listen
3. Binäre Suche
4. Sortieralgorithmen
5. Generics

Interfaces

Heinrich Heine Universität Düsseldorf

Was sind Interfaces?

- können von Klassen implementiert werden
- schreiben Klassen vor, welche Methoden sie besitzen müssen
- eine Klasse kann beliebig viele Interfaces implementieren
- ein Objekt A, welches ein Interface B implementiert, ist eine Instanz von A, sowie auch von B

Beispiel für ein Interface:

```
interface Sortable{
    double weight();
}
```

[1]

[1] *public* ist in Methoden, die von einem Interface vorgeschrieben werden nicht nötig

Interface implementieren

Wie implementiert man ein Interface?

```
    public class Item implements Sortable{
        //inhalt der Klasse
    }
```

Nun muss die Klasse Item eine Methode *weight()* implementieren, die einen *double* zurück gibt.
Wir schreiben nun also vor, dass jedes Objekt, welches eine Instanz von Sortable ist, eine bestimmte Logik hat

Interface Klasse A

```
class Schwert implements Sortable{
    String name;
    static final double WEIGHT = 7;
    Schwert(String name){
        this.name = name;
    }
    double weight(){
        return WEIGHT;
    }
}
```

Interface Klasse B

```
class Heiltrank implements Sortable {
    String name;
    int count;
    static final double WEIGHT = 2.5;
    Heiltrank(String name, int count){
        this.name = name;
        this.count = count;
    }
    double weight(){
        return count * WEIGHT;
    }
}
```

Interfaces

Heinrich Heine Universität Düsseldorf

Da Objekte der Klasse A, wie auch Objekte der Klasse B ebenfalls Instanzen von *Sortable* sind können wir diese nun auch so speichern. Wir können also ein Array von *Sortable* Objekten anlegen, und deshalb darauf vertrauen, dass jedes Objekt die Methode *weight()* implementiert.

```
    Sortable[] inventory = new Sortable[3*9];
    inventory[0] = (Sortable) new Schwert("Excalibur");
```

Wobei hilft uns das noch?

Gliederung

1 Interfaces

2 Listen

3 Binäre Suche

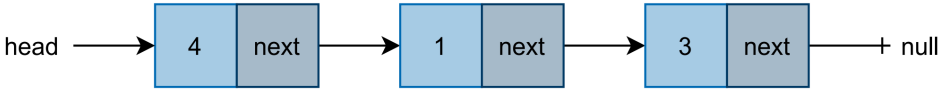
4 Sortieralgorithmen

5 Generics

Listen

Warum Listen?

- erste Node wird *head* genannt (letzte wird *tail* genannt)
- jede Node hat einen Zeiger auf nächste Node
- sie sind dynamisch vergrößerbar
- effizient, wenn wir einfügen oder löschen wollen



Listentypen

- **Einfach verkettete Liste:** Jede Node zeigt nur auf die nächste Node.
- **Doppelt verkettete Liste:** Jede Node zeigt auf vorherige und nächste Node.

Node-Klasse

```
class LinkedList {
    private Node head;

    private class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }
    (...)
}
```

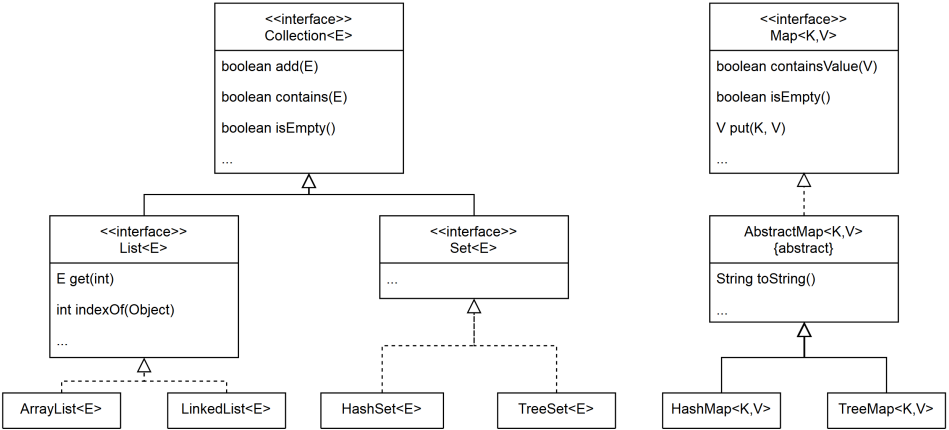
Einfügen am Anfang

```
1 void insertAtHead(int value) {
2     Node newNode = new Node(value);
3     newNode.next = head;
4     head = newNode;
5 }
```

Einfügen am Ende

```
void insertAtTail(int value) {
    Node newNode = new Node(value);
    if(head == null) {
        head = newNode;
        return;
    }
    Node current = head;
    while(current.next != null) {
        current = current.next;
    }
    current.next = newNode;
}
```

Übersicht des Collection-Frameworks



Gliederung

1 Interfaces

2 Listen

3 Binäre Suche

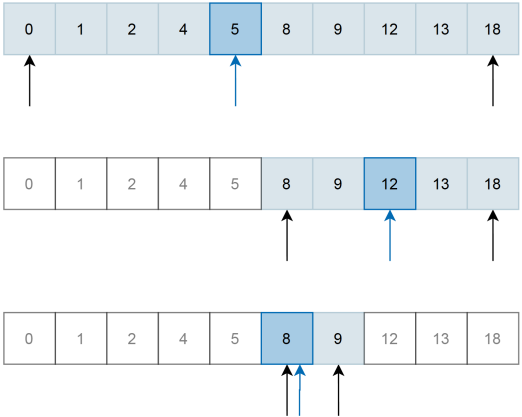
4 Sortieralgorithmen

5 Generics

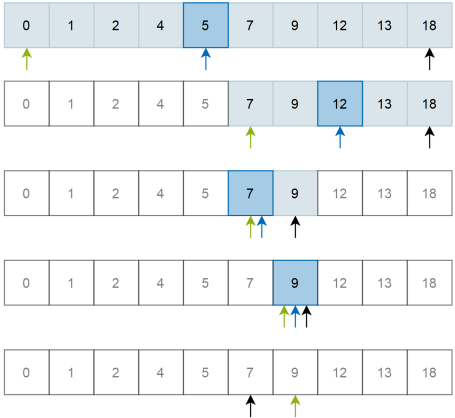
Binäre Suche - Ziel

- Ziel: Finde die Position eines bestimmten Elements in einem sortierten Array.

Beispiel: Suche nach der Zahl 8



Beispiel: erfolglose Suche nach der Zahl 8



Binäre Suche - Beispiel

```
int[] myArray = {1,4,5,6,8,8,10};
int gesuchteZahl = 5;
```

1. Mittleres Element: 6 (Index 3)
2. 5 < 6, also suchen wir im linken Teil: {1, 4, 5}
3. Mittleres Element: 4 (Index 1)
4. 5 > 4, also suchen wir im rechten Teil: {5}
5. Mittleres Element: 5 (Index 2)
6. Zahl gefunden! Index: 2

Binäre Suche Aufgaben

- wie viele Iterationen benötigen wir im schlechtesten möglichen Fall?
- Implementieren Sie Binäre Suche
- für schnelle: können sie das auch Rekursiv implementieren?

Gliederung

1 Interfaces

2 Listen

3 Binäre Suche

4 Sortieralgorithmen

5 Generics

Sortieralgorithmen

Heinrich Heine Universität Düsseldorf

Um Binäre Suche anwenden zu können muss unser Array jedoch bereits sortiert sein. Dafür benutzen wir Sortieralgorithmen. Welche kennen sie aus der Vorlesung?

Sortieralgorithmen

Heinrich Heine Universität Düsseldorf

Um Binäre Suche anwenden zu können muss unser Array jedoch bereits sortiert sein. Dafür benutzen wir Sortieralgorithmen. Welche kennen sie aus der Vorlesung?

- Insertion Sort
- Merge Sort

Insertion Sort - Funktionsweise

Heinrich Heine Universität Düsseldorf

1. Starte mit dem zweiten Element der Liste.
2. Vergleiche das aktuelle Element mit den Elementen der sortierten Teilliste (links davon).
3. Füge das aktuelle Element an der richtigen Position in die sortierte Teilliste ein.
4. Wiederhole die Schritte 2-3 für alle verbleibenden Elemente.

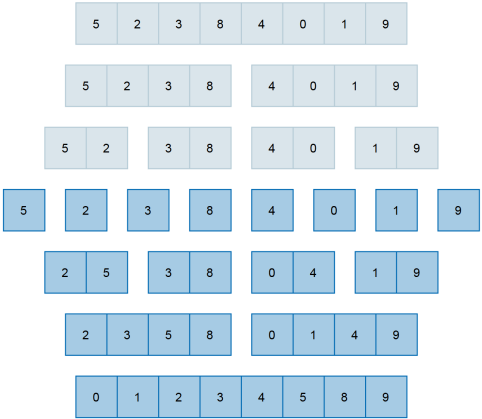
Insertion Sort - Beispiel

5	2	3	8	0	4
2	5	3	8	0	4
2	3	5	8	0	4
2	3	5	8	0	4
0	2	3	5	8	4
0	2	3	4	5	8

Merge Sort - Funktionsweise

1. Teile die Liste rekursiv in zwei Hälften, bis jede Teilliste nur noch ein Element enthält.
2. Sortiere die Teillisten, indem du sie paarweise vergleichst und zusammenführst.
3. Wiederhole den Zusammenführungsprozess, bis eine vollständig sortierte Liste entsteht.

Merge Sort - Beispiel



Gliederung

1 Interfaces

2 Listen

3 Binäre Suche

4 Sortieralgorithmen

5 Generics

Generics

Problem:

- Wenn wir eigene Datenstruktur nun mit neuer eigener Klasse verwenden wollen müssen wir unsere Datenstruktur erneut implementieren.

```
    public IntList list = new IntList();
    list.add(new Person("Superman"));          //funktioniert nicht
```

Lösung:

- wir machen aus den festen Datentypen Variablen

Generics

```
public class LinkedList<T>{
    private class Node{
        private T element;
        private Node next;
        private Node(T element, Node next){
            this.element = element;
            this.next = next;
        }
    }
}
```

Wir ändern den festen Datentypen nur auf T. Damit wird dieser variabel. Gleiches machen wir bei allen Methoden

Generics - Verwendung

Unsere Generische Liste können wir nun mit allen Datentypen verwenden, die wir benutzen wollen:

```
LinkedList<String> stringList = new LinkedList<String>();
LinkedList<Person> personList = new LinkedList<Person>();
```

Achtung: primitive Datentypen können wir **nicht** als Generic verwenden:

```
LinkedList<int> intList = new LinkedList<int>();
```

Jedoch existiert die Wrapper Klasse Integer. Folgendes funktioniert also:

```
LinkedList<Integer> intList = new LinkedList<Integer>();
```

Beispielaufgabe

1. Schreiben Sie eine Klasse *Stack*, die einen *int Stack* als Datenstruktur implementiert
2. implementieren Sie *push*
3. implementieren Sie *pop*
4. implementieren Sie *peek*
5. implementieren Sie *isEmpty*
6. implementieren Sie *size*
7. ändern Sie ihre Klasse so, dass diese generisch funktioniert